# DECKARD'S ORCHESTRATOR — Handoff

## What this is

A generative music program. It writes and plays complete songs — drums, bass, chords, melodies — in eight genres (lofi, house, citypop, jungle, dungeon, wise, barber, ambient). It also samples: drop in an audio file and it will chop it and play it as part of the band.

It ships as **one HTML file**. Not a website, not an app with a server: open `player.html` in a browser, from a USB stick, on a plane, and it works. There are no `<script src>` tags, no CDN calls, no network requests — every library, every corpus of musical data, the whole engine, is embedded in that file. It is about 1 MB and contains 204 functions and five data corpora. That constraint is deliberate: the person it is built for is a hardware musician, not a sysadmin.

**The goal:** a machine that improvises real music, not a random note generator. Every part must be defensible as music, and every claim about it must be measured rather than asserted.

---

## The prime directive

**Nothing is baked in except the physics of music theory.**

Music here is two layers:

- **HARD constraints — physics.** These REJECT. What a chord is; voices within an octave of their neighbours; non-chord tones resolve by step; each role has a register floor and ceiling. Invalid material is thrown away, not nudged.
- **SOFT tendencies — convention.** These WEIGHT randomness. Cadence distributions, root-motion strength, 8-bar entrance boundaries, genre leanings. "Often", never "always".

If you find yourself writing a lookup table of musical *content* — a fixed chord voicing, a hardcoded intro lineup, a per-genre riff — you have violated the architecture. Ask: **could a different seed produce a different valid result here?** If not, you baked something in.

The seed explores. The constraints referee.

---

## Running and building

```
node tests.js          # the standing suite — 207 checks, run before EVERY ship
node print_roll.js 12  # print a song's notes and READ them (see below)
python3 bundle.py      # modules -> /tmp/engine_bundle.js  (browser scope)
python3 make_player.py # bundle + synth + libraries + corpora -> player.html
```

Edit the numbered modules. **Never hand-edit `player.html` — it is generated, and your change will vanish on the next build. After changing any module you must re-run `bundle.py` *and* `make_player.py`; forgetting the first is a recurring bug (the player silently runs stale engine code).**

---

## THE TEST THAT MATTERS

```
node print_roll.js <seed> <bars>
```

**This is the most important test in the project and it must always be run.**

It prints a song's actual notes: pitched parts as a 16-step grid per bar, drums as separate lanes, with rests and section boundaries marked.

```
── DRUMS (drums)  enters bar 1
  bar 1:
        kick       O...............
        snare      ........O.......
        closedHat  o.......o.......

── BASS (bass)  enters bar 1
  bar 1 |C2 | . | . | . | . | . |C2 | . |G2 | . | . | . | . | . | . | . |
```

**Why it is the only test that really counts.** Statistics agree with themselves. Over and over on this project, aggregate numbers reported success while the music was broken:

- The suite passed "chords are tight" while every chord sat *below the bass* — because it measured spans and gaps, never absolute register.
- The suite reported drums were fine while a bar held two hi-hats and nothing else. Reading the roll found it in seconds.

- A “96% in key” score told us nothing about whether a bar was music.

**You can hear by reading.** Pitch, rhythm, register, spacing, repetition, density — all visible in the note data. The one thing you genuinely cannot judge from notes is timbre and mix. “I can’t hear it” is not an excuse for skipping the roll; it is the reason the roll exists.

**Read it before and after every change.** If a human would see the problem in a picture of the notes, you must check for it in the data.

Two hard-won cautions:

1. **The printer itself can lie.** An early version collapsed stacked drum hits onto one row, hiding the kick under the hat, and I misread a working groove as broken. Drums now print as separate lanes and simultaneous pitches are stacked.
2. **Probe where the music actually is.** The chord texture staggers its onsets across steps 0–6 on purpose. Sampling at step 2 catches a chord half-arrived and calls it “not a chord” — a fault in the measurement, not the music.

---

## The standing suite

`tests.js` — 207 checks, each measured across 40 seeds. It exists because every bug that ever shipped became a permanent test. Run it before every ship; a “fix” has broken other invariants more than once and the suite caught it.

What it guards, in groups:

- **Physics** — chords above the bass, lead above the chords, register floors, no unison collisions, notes in key.
- **The loop laws** — the bass and lead repeat per loop; loop 2 repeats loop 1 exactly; loop 3 DEPARTS; no exact loop plays three times.
- **Story** — songs rise from their opening, the payoff lands late, drops fall from a height, sections differ (2-loop rule), no lineup runs three sections unchanged.
- **Arrangement** — every song opens with 2+ elements, nothing sounds thin more than 4 s, the kit arrives within half a loop.
- **External referees** — `tonal` names every sounding chord (100%), `pitchfinder` agrees with our YIN (12/12), the YM2612 port matches the C emulator sample-for-sample, the Clouds reverb decays.
- **Sampling** — onsets land on transients, chops start on attacks, time-stretch holds

pitch, the load path cannot regress.

---

## Architecture

Modules run in order; each is small and does one thing.

| Module | Role |
|---|---|
| `00_theory.js` | Pitch physics: scales, degrees, chord tones, folding. |
| `01_listening.js` | The shared ears. Reads notes already played → implied harmony, rhythm, register space, phrase grid, motifs, density. Everything "hears" through this. |
| `02_engine_base.js` | Seeded RNG, `pick` / `wpick`, and the HARD/SOFT constraint sets. |
| `03_engines_pair.js` | **Harmony** and **bass**. Voicing by constraint satisfaction; the stored chord texture; loop discipline. |
| `05_drum_engine.js` | Full GM kit, feels (four/boombap/breaks/sparse), ABACAAD phrasing, ghost notes, fills, amen-style resequencing. |
| `06_melody_engine.js` | A **variable ensemble** — any number of leads, counters, pads, arps, a dueling lowline. Never "the lead". |
| `07_conductor.js` | Layer 0. Key, tempo, meter, form, genre, focal voice — and draws material from the corpora. |
| `08_ghost.js` | Runs after EVERY engine entry. Reads the actual notes and corrects cross-engine violations. Its invariants are tested at zero. |
| `09_theme.js` | The song's protagonist motif and its transformations (sequence, inversion, augmentation, fragmentation…). |
| `10_groove.js` | Swing and micro-timing for every part at once. Per-genre, drawn within researched ranges. |
| `11_slicer.js` | Audio sampling: onset detection, pad banks, the flip, beat/downbeat tracking, time-stretch. |
| `12_sample_notes.js` | YIN pitch, chromagram, Krumhansl-Schmuckler key, chord recognition. |

| | |
|---|---|
| `13_flip.js` | Symbolic sampling: chops phrases into pads, and **recombines dimensions across sources**. |
| `04_pipeline.js` | Runs the engines in random order, calls the ghost, arranges the song, applies theme development and groove. |
| `synth.js` | Web Audio: instruments, drums, YM2612 FM, modal resonator, pump, reverb, offline render. |

### Two ideas that hold it together

**Order independence.** The engines run in a random order every song. Whoever goes first GENERATES from the chart; everyone after READS the listening layer and FOLLOWS. Nothing about who opens is hardcoded.

**Entrances vs arrangement are different things.** When a part becomes *available* and who actually *plays* in a chapter are separate decisions. Conflating them is what made every intro one sparse instrument for 18 seconds.

---

## The corpora

Five bodies of musical data are embedded. All of it is stored **relatively** — scale degrees, scale-steps, durations, drum lanes — never absolute pitch. That is the whole trick: material from a piece in E-flat drops into a song in F# with nothing to correct, and cannot land out of key.

| Corpus | Contents | Source |
|---|---|---|
| `corpus.json` | 400 themes, 320 grooves, 200 textures, 280 basslines | our own engines, harvested from 10,000 songs |
| `jazz_corpus.json` | 600 progressions, 200 turnarounds, harmonic rhythm, form | 2,817 Real Book lead sheets — **chords only, no melodies** |
| `folk_corpus.json` | 900 melodic phrases, 300 openings | 1,032 traditional tunes (public domain — melodies free to take) |
| `bach_corpus.json` | 400 progressions, 700 voice lines (all 4 voices) | 39 Bach chorales |
| | | |

| | | |
|---|---|---|
| `dimensions.json` | 500 rhythms × 700 contours = 350,000 combinations | all of the above, **split apart** |

**Vocabulary vs grammar.** The corpus is not "play jazz". Lofi, city pop and barber beats are jazz-*influenced* with their own rules. So progressions are *characterised* — seventh-chord share, functional pull, stepwise motion, whether they loop or cadence — and each genre states what it wants. Measured: citypop draws 52% seventh chords, dungeon 23%, from the same 600 progressions.

**Cross-source recombination.** Rhythm and pitch are independent: *when* a note happens has nothing to do with *which way* it moves. Split apart, 1,217 harvested phrases become 350,000 combinations. A lead can be the rhythm of a Bach chorale carrying the contour of a folk tune over a jazz progression. This only works on notes — you cannot lift the rhythm of a recording away from its pitch.

A recombined part is **not exempt from the laws**. The recombined bass still states the chord root, still develops across loops, and is skipped entirely when it cannot do the job (unknown changes, or two chords per bar).

Rebuild any corpus with `node build_corpus.js`, `build_jazz.js`, `build_bach.js`, `build_folk.js`, then `build_dimensions.js`. Harvesting sets `IMPROV_HARVEST`, which forbids the engines from drawing on the corpus — otherwise the library feeds on itself and narrows.

---

## Working on this

**Read the project text files every round.** `Chords`, `Melody 1-3`, `Drums`, `Rule_of_3`, `2_bar_rule`, `8bar_loop_escape`, `Sections_of_a_Song`, `Theme`, `Zelda`. They are the spec and they are authoritative. Weeks were lost with code contradicting a file nobody had read.

**Research before designing.** Cadence frequencies, counterpoint rules, GM drum maps, onset detection, sampling technique — all findable, and finding them fixed things guessing had broken.

**Measure across all seeds (≥40) and report only that number.** One good chord shown as proof while 91% were broken is how trust was lost here.

**Never claim fixed until the measurement moves.** If it didn't move, say so. If you changed the test to make it pass, you lied even though it is green.

**Delete old code when you replace it.** Verify exactly one of each function after any

splice. Buried duplicates kept "fixed" things broken for weeks.

**Ship to `/mnt/user-data/outputs` after every working change.** The sandbox resets.

### Failure modes that actually happened

- Reporting statistics instead of reading the roll. The single most expensive habit on this project.
- Editing a module and not re-running `bundle.py`, so the player ran stale code.
- String replacements that silently didn't match, leaving the code unchanged while the summary claimed otherwise. **Assert that every edit applied.**
- Asserting a limit instead of testing it ("we can't use that library") when npm was reachable the whole time.
- Building something that wasn't asked for because it seemed impressive.

---

## Known unfinished

- **Drums have no real-music corpus.** Everything harvested is pitched. Google's Groove MIDI Dataset (CC-BY, 13.6 hours of human drumming) is the right source; it is blocked from this sandbox but downloadable elsewhere.
- **Voice leading sits at ~52% stepwise against Bach's measured 77.3%.** Wiring a preference barely moved it: every candidate voicing is a close-position stack near the same centre, so there is nothing better to choose. Fixing it needs genuinely different inversions and spacings generated.
- **Melodies and grooves are not yet characterised** the way progressions are, so genre cannot select them on properties — only on major/minor.
- **The Clouds reverb port** is faithful and passes its impulse test; the YM2612 port matches the C emulator exactly. Both could be replaced by proper WASM builds (`rubberband-web` is on npm and untried).
- **Sections and phrases are not in the corpus** — only ingredients. Recombining whole arrangement states would mix and match song *structure*.